



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

Text Classification

Natural Language Processing

<http://vku.udn.vn/>

1

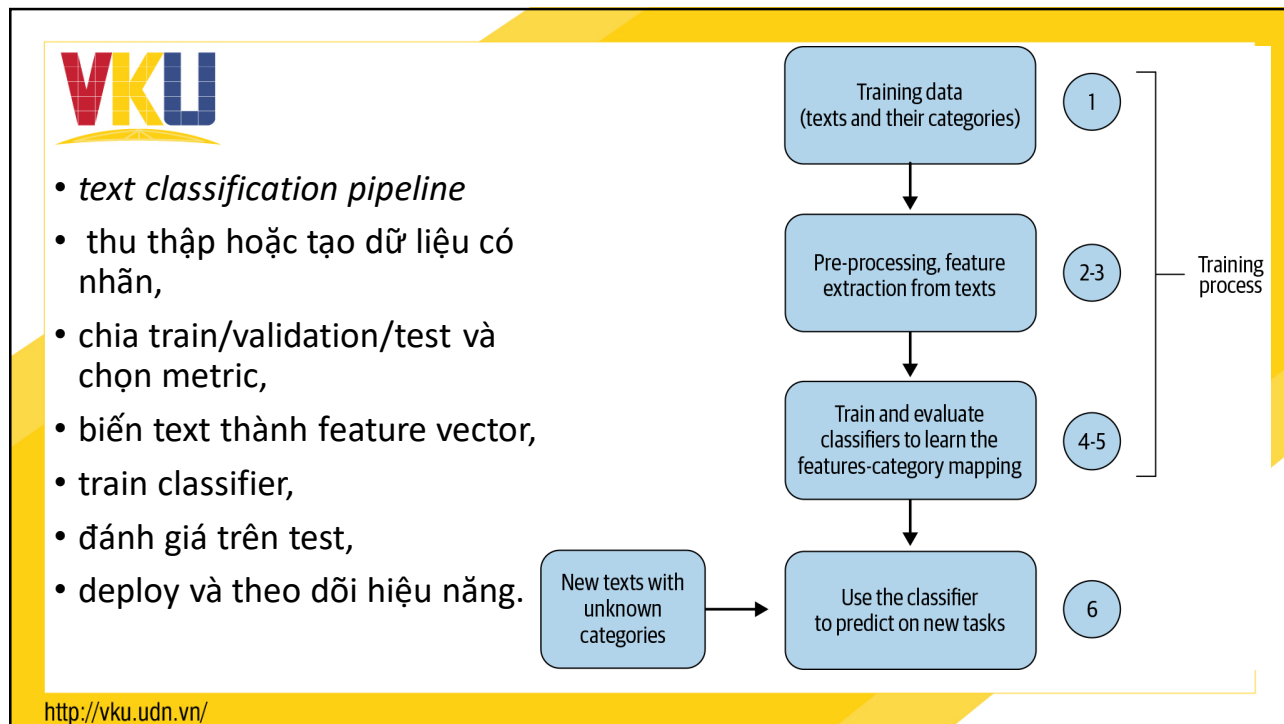


Supervised classification

- Phân loại có giám sát: Ba loại dựa trên số lượng danh mục:
 - nhị phân: phân loại email spam/không spam
 - đa lớp: phân loại cảm xúc đánh giá tích cực, tiêu cực, trung tính
 - đa nhãn: 1 bài báo thể thao có thể phân vào loại “Thể thao”, “Bóng đá”
- Phân loại văn bản đôi khi cũng được gọi là phân loại chủ đề, phân loại văn bản hoặc phân loại tài liệu. (Khác với Phát hiện chủ đề)
- Text classification / topic classification, text categorization / document categorization.
- Phát hiện ngôn ngữ; Phân loại tin giả...

<http://vku.udn.vn/>

2



3

VKU

- Không phải lúc nào cũng cần học máy ngay
 - Có thể dùng rule/lexicon-based approach làm baseline rất nhanh, ví dụ sentiment bằng danh sách từ tích cực/tiêu cực/icon.
- Với dữ liệu có nhãn, không có một classifier nào luôn tốt nhất
 - chất lượng dữ liệu và phân bố lớp ảnh hưởng cực mạnh,
 - phải thử nhiều lựa chọn,
 - phải nhìn vào confusion matrix, không chỉ nhìn accuracy.

<http://vku.udn.vn/>

4



Dữ liệu và gán nhãn

- Phải có dữ liệu và dữ liệu đó phải có nhãn phù hợp với bài toán.
- Ví dụ nếu bài toán là phân loại email spam/ham thì mỗi email phải được gán nhãn spam hoặc ham.
- Một dataset “tốt” không chỉ là nhiều mẫu, mà phải là dataset đại diện cho dữ liệu sẽ gặp ngoài thực tế.
- Chia: train/validation/test
 - train: để mô hình học
 - validation/dev: để so sánh mô hình, tune tham số
 - test: để đánh giá cuối cùng

<http://vku.udn.vn/>

5



Trích xuất đặc trưng

- BoW
- TF-IDF
- word embeddings
- document embeddings
- hoặc các biểu diễn đầu vào cho deep learning
 - mô hình không học trực tiếp trên chuỗi ký tự thô, mà học trên **vector biểu diễn** của văn bản

<http://vku.udn.vn/>

6



Train a classifier

- feature vector + nhãn $\rightarrow$ train mô hình phân loại.
- không có classifier nào luôn tốt nhất cho mọi bài toán.
 - Trong thực tế, phải thử nhiều hướng rồi so sánh.
- giữ nguyên pipeline nhưng thay đổi mô hình như Naive Bayes, Logistic Regression, SVM.
- Benchmark on the test set
 - accuracy
 - precision / recall / F1
 - confusion matrix

<http://vku.udn.vn/>

7



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

Model Improvement

Natural Language Processing

<http://vku.udn.vn/>

8



Cải tiến mô hình

- luôn giữ **cùng train/test split**
- luôn giữ **cùng cách đánh giá**
- mỗi lần chỉ đổi **một yếu tố chính**
- so sánh với **baseline hiện tại**
- Ví dụ baseline hợp để so sánh
 - **SVM + TF-IDF**: mô hình tốt nhất hiện tại
 - thêm **Logistic Regression + TF-IDF** hoặc **Naive Bayes + BoW** làm đối chiếu
- Mọi hướng cải tiến sẽ trả lời câu hỏi:
“Đổi yếu tố này thì precision / recall / F1 của lớp 1 thay đổi ra sao?”

<http://vku.udn.vn/>

9



Hướng 1: Giảm số lượng đặc trưng

- Giảm số chiều của vector để:
 - bớt nhiễu
 - bớt độ thưa (sparse)
 - xem mô hình có học ổn hơn không
 - đôi khi giúp lớp thiếu số tốt hơn
- Với BoW hoặc TF-IDF, thử giới hạn số đặc trưng bằng max_features.
- Ví dụ thử các mức:
 - 1000
 - 3000
 - 5000
 - 10000
 - không giới hạn

<http://vku.udn.vn/>

10



Hướng 1: Giảm số lượng đặc trưng

- **Kiểm tra điều gì?**

- feature quá nhiều có làm mô hình nhiễu không?
- giảm feature có làm recall lớp 1 tăng không?
- F1 có cải thiện không?

- **Kỳ vọng**

- Có thể accuracy không tăng nhiều
- Nhưng F1 lớp 1 hoặc recall lớp 1 có thể tốt hơn
- Hoặc thời gian train nhanh hơn

- **Nhận xét**

- khi giảm xuống 5000 feature, mô hình bắt lớp 1 tốt hơn
- khi giảm quá thấp, mô hình mất thông tin, F1 lại giảm
- vậy tồn tại một mức feature “vừa đủ tốt”

<http://vku.udn.vn/>

11



Hướng 2: Xử lý mất cân bằng lớp

- **Mục tiêu**

- Dữ liệu của mình đang lệch lớp khá rõ, nên mô hình dễ thiên về lớp 0.
- Hướng này nhằm giúp mô hình **chú ý hơn đến lớp 1**.

- **Cách A: dùng class_weight**

- Với: Logistic Regression & SVM
- so sánh:
 - không dùng class_weight
 - dùng class_weight="balanced"

- **Cách B: resampling**

- Có thể thử:
 - **undersampling** lớp 0
 - **oversampling** lớp 1

<http://vku.udn.vn/>

12



Hướng 2: Xử lý mất cân bằng lớp

- **Kiểm tra điều gì?**
 - recall lớp 1 có tăng không?
 - confusion matrix có bớt bỏ sót lớp 1 không?
 - precision có bị giảm bao nhiêu?
- **Kỳ vọng gì?**
 - Khi xử lý imbalance:
 - recall lớp 1 thường tăng
 - nhưng precision có thể giảm
 - accuracy có thể không tăng, thậm chí giảm nhẹ
- → Một cải tiến **không nhất thiết làm accuracy tăng**, nhưng vẫn có thể làm mô hình tốt hơn cho mục tiêu bài toán.
- **Nhận xét**
 - sau khi dùng `class_weight="balanced"`, mô hình phát hiện lớp 1 tốt hơn
 - số mẫu lớp 1 bị bỏ sót giảm
 - dù accuracy giảm nhẹ, F1 lớp 1 lại tăng, nên cải tiến này là có ích

<http://vku.udn.vn/>

13



Hướng 3: So sánh BoW và TF-IDF

- **Mục tiêu**
 - Xem cách biểu diễn văn bản ảnh hưởng thế nào đến mô hình.
- **Cần làm gì**
 - Giữ nguyên:
 - cùng input
 - cùng train/test split
 - cùng model
 - Rồi so sánh:
 - BoW
 - TF-IDF

<http://vku.udn.vn/>

14



Hướng 4: Thử n-gram

• Mục tiêu

- Thêm thông tin về **cụm từ**, không chỉ từng từ đơn lẻ.
- Trong bài báo tiếng Việt, nhiều tín hiệu phân loại nằm ở cụm như:
- thị trường, doanh nghiệp, lãi suất, bất động sản, đội tuyển, phòng vé
- Nếu chỉ dùng unigram, mô hình có thể mất bớt ngữ cảnh.

• Cần làm gì

- So sánh:
 - unigram: `ngram_range=(1,1)`
 - unigram + bigram: `ngram_range=(1,2)`

<http://vku.udn.vn/>

15



Hướng 4: Thử n-gram

• Kiểm tra điều gì?

- bigram có giúp nhận diện topic tốt hơn không?
- precision / recall lớp 1 có cải thiện không?
- số feature có tăng quá mạnh không?

• Kỳ vọng

- bigram thường giúp mô hình hiểu cụm từ tốt hơn
- nhưng cũng làm số chiều tăng mạnh
- nếu dữ liệu không đủ hoặc không xử lý tốt, có thể sinh nhiễu

• Nhận xét

- thêm bigram giúp mô hình nhận tốt hơn các cụm từ chuyên ngành kinh tế
- F1 lớp 1 tăng nhẹ
- nhưng thời gian train tăng và số feature lớn hơn

<http://vku.udn.vn/>

16



Hướng 5: Thử đầu vào khác nhau

• Mục tiêu

- Xem phần nào của bài báo hữu ích nhất cho phân loại:
 - chỉ tiêu đề?
 - tiêu đề + mô tả?
 - toàn bộ nội dung?

• Cần làm gì

- So sánh 3 đầu vào:
 - input_title
 - input_title_desc
 - input_full
- Trên cùng một pipeline mạnh, ví dụ:
 - **SVM + TF-IDF**
 - hoặc **Logistic Regression + TF-IDF**

<http://vku.udn.vn/>

17



Hướng 5: Thử đầu vào khác nhau

• Kiểm tra điều gì?

- title có đủ thông tin không?
- thêm desc có giúp không?
- dùng full text có thật sự tốt hơn không?

• Kỳ vọng

- title thường nhanh, gọn, nhưng thiếu ngữ cảnh
- title + desc nhiều khi là điểm cân bằng rất tốt
- full text có thể mạnh hơn, nhưng cũng dễ mang nhiều nhiễu

• Nhận xét

- chỉ dùng title cho kết quả thấp hơn vì thông tin quá ngắn
- title + desc cải thiện rõ rệt
- full text không tăng nhiều hoặc thậm chí giảm vì mang nhiều thông tin thừa

<http://vku.udn.vn/>

18



Hướng 6: Điều chỉnh tham số mô hình

- **Mục tiêu**
- Không dùng đúng cấu hình mặc định mại, mà thử vài tham số cơ bản để xem mô hình thay đổi ra sao.
- **Cần làm gì**
 - **Với Naive Bayes**
 - Thử alpha:
 - 0.1
 - 0.5
 - 1.0
 - 2.0
 - alpha là hệ số smoothing → ảnh hưởng đến cách mô hình xử lý từ hiếm.
 - **Với Logistic Regression**
 - Thử C:
 - 0.1
 - 1
 - 10
 - C điều khiển mức regularization:
 - C nhỏ → regularization mạnh hơn
 - C lớn → mô hình “thoáng” hơn
 - **Với SVM**
 - Thử C:
 - 0.1
 - 1
 - 10
 - Với LinearSVC, C cũng ảnh hưởng tới ranh giới phân loại.

<http://vku.udn.vn/>

19



Hướng 6: Điều chỉnh tham số mô hình

- **Kiểm tra điều gì?**
 - tham số nào giúp F1 lớp 1 tốt hơn?
 - mô hình có overfit / underfit không?
 - precision và recall đổi theo hướng nào?
- **Kỳ vọng**
 - có thể cải thiện không quá lớn
 - nhưng giúp sinh viên hiểu mô hình không phải “hộp đen”
- **Nhận xét**
 - khi tăng C, SVM phân lớp mạnh tay hơn nhưng precision giảm
 - khi điều chỉnh alpha, Naive Bayes ổn định hơn với từ hiếm
 - tham số tốt nhất phụ thuộc dữ liệu

<http://vku.udn.vn/>

20



Hướng 7: Xử lý với từ tiếng Việt

• Mục tiêu

- Tiếng Việt có nhiều từ **ghép nhiều âm tiết**, ví dụ:
 - kinh tế, thể thao, bất động sản, doanh nghiệp, lãi suất, thị trường
- Nếu mình tách theo khoảng trắng đơn giản, mô hình sẽ thấy:
 - kinh và tế
 - thể và thao
 - thay vì thấy cả cụm ý nghĩa → làm mất thông tin.

• Cần làm gì

- Thử một bước **word segmentation / tokenization tiếng Việt** để biến:
 - kinh tế → kinh_tế
 - thể thao → thể_thao
 - bất động sản → bất_động_sản
- rồi mới đưa vào BoW / TF-IDF.
- Dùng công cụ tách từ tiếng Việt để tokenize đúng hơn trước khi vectorize.
 - Ví dụ: dùng Underthesea

<http://vku.udn.vn/>

21



Hướng 7: Xử lý với từ tiếng Việt

• Kiểm tra điều gì?

- mô hình có hiểu cụm từ tiếng Việt tốt hơn không?
- precision / recall / F1 có tăng không?
- các từ khóa chủ đề có trở nên rõ hơn không?

• Kỳ vọng

- Hướng này thường có tiềm năng cải thiện tốt, vì nó tác động trực tiếp đến chất lượng biểu diễn văn bản.
- Đặc biệt với bài báo tiếng Việt theo topic, các cụm nhiều âm tiết là tín hiệu rất quan trọng.

• Nhận xét

- sau khi gộp từ ghép tiếng Việt, vectorizer giữ được nghĩa của các cụm quan trọng hơn
- mô hình phân biệt topic tốt hơn
- F1 lớp 1 cải thiện so với cách tách từ đơn thuần theo khoảng trắng

<http://vku.udn.vn/>

22



23



24

